

FlaskGuard

Custom Web Application Firewall

Technical Report — Part 5

Reverse Proxy, Demo Backend Application, & Full Test Suite

Repository	<code>github.com/echoenvoy/FlaskGuard</code>
Report Part	5 of 5
Author	Amhidi Hamza
Key Modules	<code>proxy.py</code> , <code>demo_backend/</code> , Test Suite
Date	June 2026

Contents

1	Introduction	2
1.1	Recap of the Series	2
1.2	Scope of This Report	2
2	Reverse Proxy — proxy.py	2
2.1	Role of the Reverse Proxy in the Architecture	2
2.2	Request Forwarding Semantics	3
2.3	Header Forwarding Strategy	3
2.3.1	Hop-by-Hop Headers	3
2.3.2	Proxy Identification Headers	4
2.4	Response Header Filtering	4
2.5	Error Handling	4
2.6	Redirect Handling	4
3	Demo Backend Application — demo_backend/	5
3.1	Purpose	5
3.2	Endpoint Design	5
3.3	The /search Endpoint as a Vulnerability Surface	5
3.4	Network Isolation via Docker	6
4	Test Suite	6
4.1	Testing Philosophy	6
4.2	Unit Tests	6
4.2.1	Detection Rule Tests	6
4.2.2	File Scanner Tests	7
4.2.3	IP Management Tests	7
4.3	Integration Tests	7
4.3.1	Attack Blocking Tests	7
4.3.2	Forwarding Tests	7
4.3.3	Rate Limiting Test	8
4.4	Administrative API Tests	8
4.5	Running the Test Suite	8
5	Complete System Architecture	8
5.1	End-to-End Request Processing Pipeline	8
5.2	Architecture Diagram	9
6	Conclusion	9

Introduction

Recap of the Series

The four preceding parts of this report series established the core security inspection and enforcement infrastructure of FlaskGuard:

- **Part 1** defined the Flask application scaffold, centralised configuration management, per-IP rate limiting, and the administrative dashboard skeleton.
- **Part 2** introduced regex-based attack detection across five vulnerability categories, structured attack logging, and Docker containerisation.
- **Part 3** added the Naive Bayes machine-learning classifier and the IP reputation system responsible for strike tracking and automated banning.
- **Part 4** implemented the seven-layer file upload scanner, addressed HTTPS and TLS architecture, and enforced secrets hardening through environment-variable overrides.

While those components constitute a sophisticated inspection engine, they require two additional elements to form a complete, deployable product: a mechanism to forward approved traffic to a real backend application, and a validated test environment that confirms every security layer operates correctly. Part 5 delivers both.

Scope of This Report

This document covers the following:

1. **The Reverse Proxy (`proxy.py`):** design rationale, forwarding semantics, header management strategy, error handling, and redirect behaviour.
2. **The Demo Backend Application (`demo_backend/`):** purpose, endpoint design, intentional vulnerability surface for testing, and Docker network isolation.
3. **The Test Suite:** testing philosophy, category breakdown, coverage summary, and how to execute the suite.
4. **System Retrospective:** an end-to-end view of the complete request processing pipeline, a comparison with enterprise WAF products, and a phased roadmap for production hardening.

Reverse Proxy — `proxy.py`

Role of the Reverse Proxy in the Architecture

A reverse proxy is the component that bridges FlaskGuard’s inspection layer with the application it protects. It accepts HTTP requests that have passed all security checks and forwards them transparently to an upstream backend server, then relays the backend’s response back to the original client.

From the client’s perspective, it communicates directly with the backend. From the backend’s perspective, every request originates from FlaskGuard. This bidirectional transparency is what makes reverse proxying central to WAF design: the protected application requires no modification, and the client requires no awareness of the intermediary. The traffic flow is as follows:

Client $\xrightarrow{\text{HTTPS Request}}$ FlaskGuard / proxy.py Application $\xrightarrow{\text{Forwarded HTTP Request}}$ Backend

Backend Application $\xrightarrow{\text{HTTP Response}}$ FlaskGuard / proxy.py $\xrightarrow{\text{Response}}$ Client

The proxy route is registered as the catch-all route in Flask, matching any path that is not first handled by the administrative namespace. This ensures that every legitimate request — regardless of HTTP method or URL depth — is forwarded without the need for per-endpoint configuration.

Request Forwarding Semantics

When a request arrives at the proxy route, `proxy.py` constructs a new outbound HTTP request directed at the configured `backend_url`. The following elements are replicated from the original client request:

- **HTTP method:** The verb (GET, POST, PUT, DELETE, PATCH, HEAD, OPTIONS) is forwarded unchanged, ensuring that the backend’s routing logic sees the correct intent.
- **URL path and query string:** The full path and any query parameters are appended to the backend base URL verbatim.
- **Request body:** The raw body bytes are forwarded as-is, preserving JSON payloads, form data, and multipart uploads without re-encoding.
- **Cookies:** Client cookies are passed through so that session management on the backend continues to function correctly.

The forwarding timeout is configurable via `proxy.timeout_seconds` in `config.yml` with a default value of ten seconds.

Header Forwarding Strategy

Correct header forwarding requires distinguishing between two classes of HTTP headers:

Hop-by-Hop Headers

Hop-by-hop headers describe properties of the connection between two adjacent nodes (client–WAF or WAF–backend) and must not be propagated further. FlaskGuard strips the following headers before forwarding:

Header	Reason for Removal
Connection	Describes this TCP connection only
Keep-Alive	Connection persistence is renegotiated with the backend
Transfer-Encoding	The WAF re-frames the body independently
TE	Transfer-extension negotiation is local
Trailers	Trailer field names are connection-specific
Upgrade	Protocol upgrades (e.g. WebSocket) must be renegotiated
Proxy-Authenticate	Proxy authentication is not forwarded to the backend
Proxy-Authorization	Likewise not forwarded

Proxy Identification Headers

The backend must be able to determine the real client IP address and the original protocol even though, from its vantage point, all traffic originates from FlaskGuard. Three standard headers are injected by the proxy:

Header Added	Purpose
X-Forwarded-For	Carries the original client IP address
X-Forwarded-Proto	Indicates whether the client used HTTP or HTTPS
X-Real-IP	Redundant IP carrier for compatibility with some backends

The **Host** header is rewritten from the client’s host value to the backend’s hostname, extracted from `backend_url`, so that the backend’s virtual hosting configuration resolves correctly.

Response Header Filtering

Not all response headers returned by the backend are appropriate to forward to the client. The following headers are excluded from the response before it is sent:

Filtered Response Header	Reason
Content-Encoding	Flask re-encodes the response body during transmission
Transfer-Encoding	Flask manages chunked transfer independently
Connection	Hop-by-hop; must not be forwarded to the client

All other response headers — including **Content-Type**, **Cache-Control**, **Set-Cookie**, **ETag**, and custom application headers — are forwarded verbatim, preserving the backend’s full response semantics.

Error Handling

Two categories of infrastructure error can occur during proxy forwarding. Both are surfaced to the client with semantically correct HTTP status codes:

Error Condition	HTTP Status	Interpretation
Backend request exceeds timeout	504 Gateway Timeout	Backend is overloaded or hung
Backend connection refused	502 Bad Gateway	Backend process is not running
Backend returns any 5xx status	Passed through	Application-level error; client receives the

Application-level errors returned by the backend (5xx responses) are relayed unmodified. This design preserves the backend’s error semantics and avoids masking genuine application failures behind generic gateway errors.

Redirect Handling

The proxy is configured to not follow redirects automatically when making backend requests. When the backend issues a **301 Moved Permanently** or **302 Found** redirect, the redirect response is forwarded directly to the client. The client then issues a new request to the redirect target, which once again enters FlaskGuard’s inspection pipeline.

This is the architecturally correct behaviour for a WAF: silently following redirects internally would allow the redirect destination to be reached without inspection, creating a potential bypass vector. Exposing the redirect to the client ensures every request in a redirect chain is subject to full security analysis.

Demo Backend Application — `demo_backend/`

Purpose

The demo backend serves two distinct but complementary purposes:

1. **Integrated test target.** End-to-end integration tests require a real backend that the proxy can attempt to reach. The demo backend provides a realistic set of endpoints — including one intentionally designed to reflect user-controlled input — against which attack payloads can be verified as blocked before reaching the application.
2. **Out-of-the-box demonstration.** Running `docker compose up` launches both FlaskGuard and the demo backend together, enabling an immediate, self-contained demonstration of the full system without requiring any external application to be configured.

Endpoint Design

The demo backend is a minimal Flask application that simulates a typical content website. Its endpoints are intentionally representative of common real-world patterns:

Endpoint	Method	Behaviour
<code>/</code>	GET	Renders a static home page
<code>/search</code>	GET	Accepts a <code>q</code> query parameter; reflects it in the response HTML
<code>/upload</code>	POST	Accepts a multipart file upload; responds with filename and size metadata
<code>/about</code>	GET	Renders a static about page
<code>/api/products</code>	GET	Returns a JSON array of product names

The `/search` Endpoint as a Vulnerability Surface

The `/search?q=` endpoint occupies a special role in the test architecture. It intentionally reflects the raw value of the `q` parameter back into the rendered HTML response. In a real application, this pattern would constitute a reflected Cross-Site Scripting (XSS) vulnerability.

This design is deliberate: it makes FlaskGuard’s protection immediately tangible. Without the WAF, a crafted XSS payload submitted to this endpoint would render as executable script in the response body. With FlaskGuard in place, the payload is blocked at the inspection layer and never reaches the backend. The contrast between the protected and unprotected states can be demonstrated live by temporarily bypassing the WAF.

Network Isolation via Docker

The demo backend’s Docker Compose service definition uses `expose` rather than `ports` to publish port 8080. This distinction has a critical security implication:

- `expose` makes port 8080 reachable only to other services on the same Docker internal network, not to the host machine or external network.
- `ports` would additionally bind the port to the host machine, making it directly accessible from outside Docker.

By using `expose`, the demo backend is accessible exclusively through FlaskGuard, accurately replicating the intended production architecture in which the protected application is never directly reachable by end users.

Test Suite

Testing Philosophy

FlaskGuard’s test suite is structured around the principle of layered validation: each component is verified in isolation before being tested as part of the integrated system. This approach serves two goals:

- **Fault localisation.** When a test fails, the layered structure makes it possible to determine immediately whether the failure is in a detection rule, a supporting utility (such as the IP manager), or the integration between components.
- **Regression prevention.** Unit tests for individual functions run fast and provide immediate feedback during development, while integration tests confirm that changes in one layer do not silently break another.

The suite is composed of three test categories: unit tests, integration tests, and administrative API tests.

Unit Tests

Unit tests exercise individual functions in isolation, using mock objects to simulate the Flask request context without starting a server. Each detection category, scanner layer, and management function has dedicated unit test coverage.

Detection Rule Tests

For each of the five attack categories (SQL Injection, Cross-Site Scripting, Path Traversal, Command Injection, and File Injection), unit tests verify four properties:

1. **True positive coverage:** known attack payloads belonging to the category are correctly flagged with the appropriate attack type label.
2. **False positive prevention:** benign inputs representative of real user behaviour (e.g., product search queries, filenames, prose text) are not flagged.
3. **Evasion resistance:** URL-encoded and case-varied variants of known payloads are detected, confirming that the normalisation step in the detection pipeline is functioning.

4. **Body coverage:** payloads delivered in the POST request body are detected, not only those in query parameters.

File Scanner Tests

The file scanner unit tests cover all seven detection layers individually. Each layer is tested with at least one malicious file that should trigger a block and at least one benign file that should pass. Selected test scenarios include:

- A PHP script submitted with a `.jpg` extension, expected to be rejected either for dangerous file type or for a MIME type / extension mismatch.
- A valid JPEG file (identified by correct magic bytes), expected to pass all checks.
- A ZIP archive containing a highly compressible payload whose compression ratio exceeds the configured threshold, expected to be flagged as an archive bomb.
- An SVG file containing an embedded `<script>` element, expected to be rejected for active content.
- A syntactically valid SVG containing only geometric drawing elements, expected to pass.

IP Management Tests

The IP manager unit tests verify three behavioural properties:

- **Automatic banning:** after the configured number of strikes, an IP address transitions to the banned state.
- **Ban expiry:** a banned IP is automatically released after the configured ban duration has elapsed, verified using short durations combined with controlled sleep.
- **Manual unbanning:** an administrator-initiated unban immediately releases a previously banned IP, including one that would not yet have expired naturally.

Integration Tests

Integration tests use Flask’s built-in test client to send crafted HTTP requests through the full `before_request` inspection pipeline without starting a network server. They validate end-to-end system behaviour rather than individual function logic.

Attack Blocking Tests

One integration test exists for each of the five attack categories. Each test confirms that a canonical payload sent in the query string results in a `403 Forbidden` response. The response body is additionally inspected to confirm that the reported attack type matches the expected category.

Forwarding Tests

A clean request carrying a benign query string is sent through the pipeline with the backend network call intercepted by a request mock. The test confirms that the response carries a `200 OK` status code, verifying that legitimate traffic is not incorrectly blocked and that the proxy forwarding path executes without error.

Rate Limiting Test

Requests are sent in a loop until the configured per-hour limit is exceeded. The test confirms that the first over-limit request returns **429 Too Many Requests**, verifying that flask-limiter’s integration with the application is functioning correctly.

Administrative API Tests

Administrative API tests send requests to the `/admin` and `/admin/api/*` namespaces using both valid and invalid Basic Auth credentials, verifying the following:

- Unauthenticated requests to the dashboard and all API endpoints receive **401 Unauthorized**.
- Authenticated requests with the correct credentials receive **200 OK** and a response body that conforms to the expected JSON schema.
- Requests with an incorrect password are rejected with **401 Unauthorized** regardless of the username.
- The complete ban/unban lifecycle — ban an IP, verify it appears in the blacklist, unban it, verify it is absent — executes correctly via the API.

Running the Test Suite

The test suite uses `pytest` with the `pytest-mock` and `requests-mock` plugins. After installing the project’s `requirements.txt`, the entire suite is executed with a single command. Coverage reporting is available via the `pytest-cov` plugin, which generates an HTML report in the `htmlcov/` directory. Individual modules or test categories can be targeted using `pytest`’s `-k` expression syntax.

Complete System Architecture

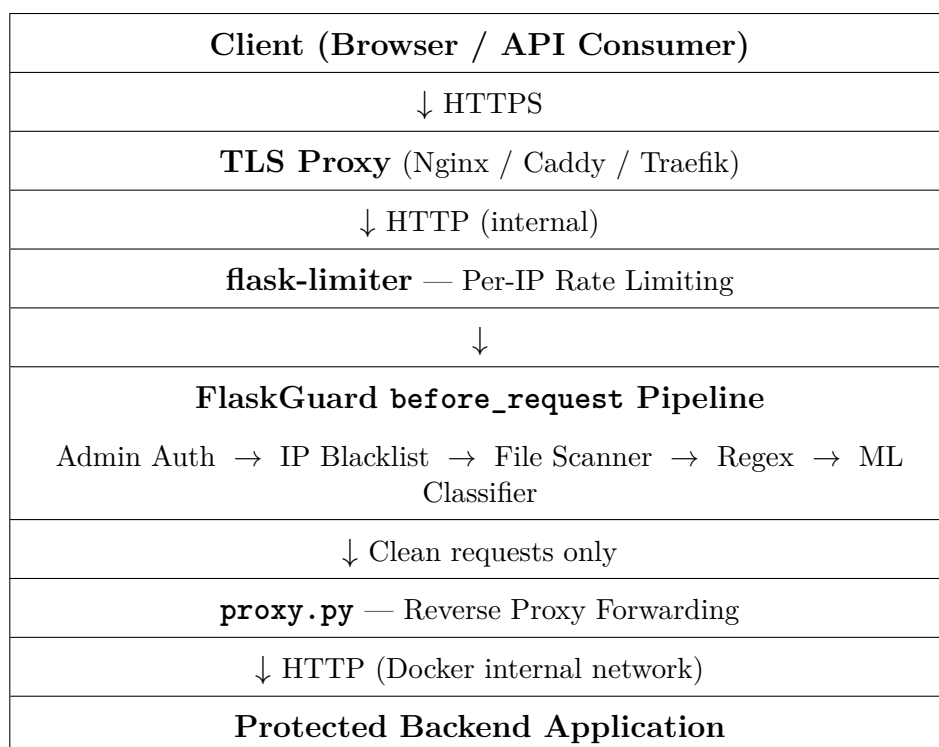
End-to-End Request Processing Pipeline

With all five components in place, every inbound request is processed through the following ordered pipeline:

1. **TLS Termination.** An external TLS proxy (Nginx, Caddy, or Traefik) terminates the HTTPS connection, decrypts the request, appends `X-Forwarded-For` and `X-Forwarded-Proto` headers, and forwards the plaintext request to FlaskGuard on its internal port.
2. **Rate Limiting.** `flask-limiter` checks the per-IP request counter against the configured daily and hourly limits. Requests that exceed the limit are rejected immediately with **429 Too Many Requests** without entering the security inspection pipeline.
3. **Administrative Authentication.** Requests targeting the `/admin` namespace are validated against the configured HTTP Basic Auth credentials. Unauthenticated requests receive **401 Unauthorized**.
4. **IP Blacklist Check.** The source IP address is looked up in the blacklist store. Banned IPs receive **403 Forbidden** immediately, without incurring the cost of further inspection.
5. **File Upload Scanning.** Multipart file upload requests are processed through the seven-layer scanner: filename validation, extension allowlisting, magic byte veri-

- fication, MIME/extension cross-check, text content analysis, SVG active-content detection, and archive bomb detection. Any layer rejection returns 403.
6. **Regex Detection.** All input fields — query string parameters, POST form fields, and request body content — are scanned against the compiled regex rule sets for all five attack categories.
 7. **ML Classifier.** If the regex layer produces no definitive match, the Naive Bayes classifier is applied as a fallback. Inputs that exceed the 70% confidence threshold for the malicious class are flagged.
 8. **Attack Logging and Strike Recording.** If either detection layer flags the request, the attack event is written to the JSON log and the plaintext security log. The source IP receives a strike; if its cumulative strike count reaches the configured threshold, it is automatically added to the blacklist for the configured ban duration. A 403 `Forbidden` response is returned.
 9. **Reverse Proxy Forwarding.** Requests that pass all preceding checks are forwarded to the backend application by `proxy.py`. The backend’s response is cleaned of hop-by-hop headers and returned to the client.

Architecture Diagram



Conclusion

Part 5 concludes the FlaskGuard series by supplying the two operational components that transform the security inspection engine into a deployable system. The reverse proxy provides transparent, standards-compliant forwarding of approved traffic to the protected backend, with correct header management, graceful error handling, and architecturally sound redirect behaviour. The demo backend gives the system a realistic test target and a self-contained demonstration environment, while its Docker network configuration faithfully models the intended production topology. The test suite validates every layer of the

system — detection rules, file scanning, IP management, proxy behaviour, and administrative API — through a combination of isolated unit tests and full-pipeline integration tests.

Taken together across all five parts, FlaskGuard represents a thorough and pedagogically coherent implementation of a Web Application Firewall. It covers the complete engineering surface of WAF design: protocol-level request handling, pattern-based and machine-learning detection, file upload security, IP reputation management, structured attack logging, containerised deployment, and administrative tooling. While it is explicitly positioned as a learning and prototyping artefact rather than an enterprise-grade product, it successfully demonstrates every foundational concept that underpins commercial WAF solutions — in a codebase that is readable, modifiable, and fully tested.